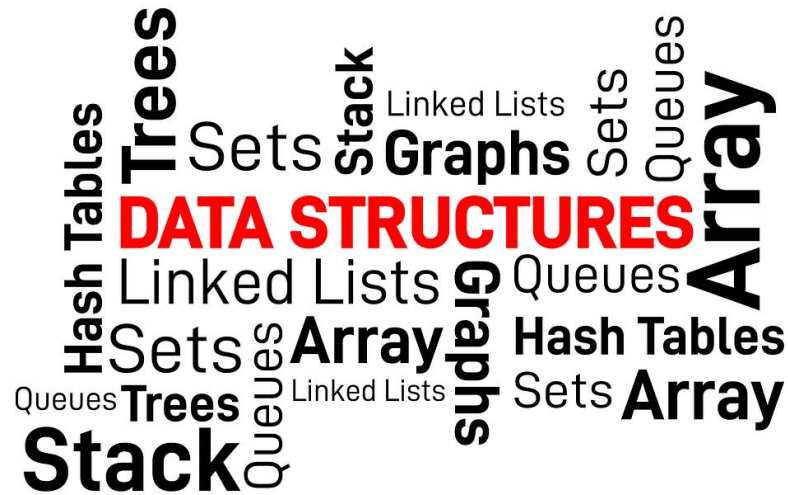
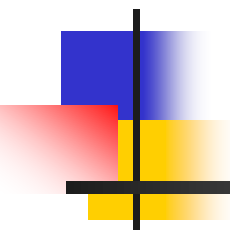




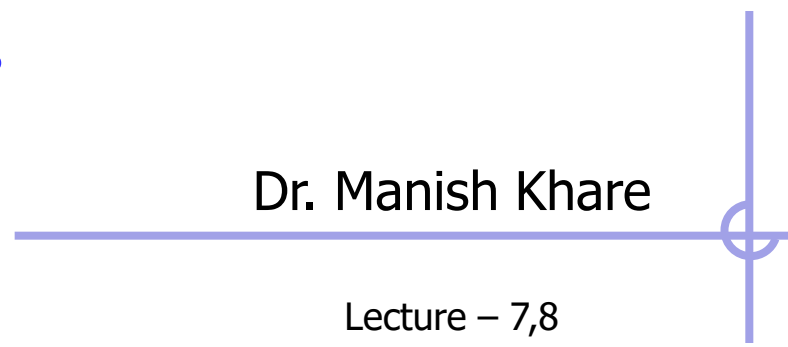
Hash Tables
Trees
Sets
Stack
Linked Lists
Graphs
Sets
Queues
DATA STRUCTURES
Linked Lists
Queues
Sets
Hash Tables
Array
Stack
Queues
Trees
Linked Lists
Graphs
Sets
Array

Data Structures

IT623

Dr. Manish Khare

Lecture – 7,8





Stacks

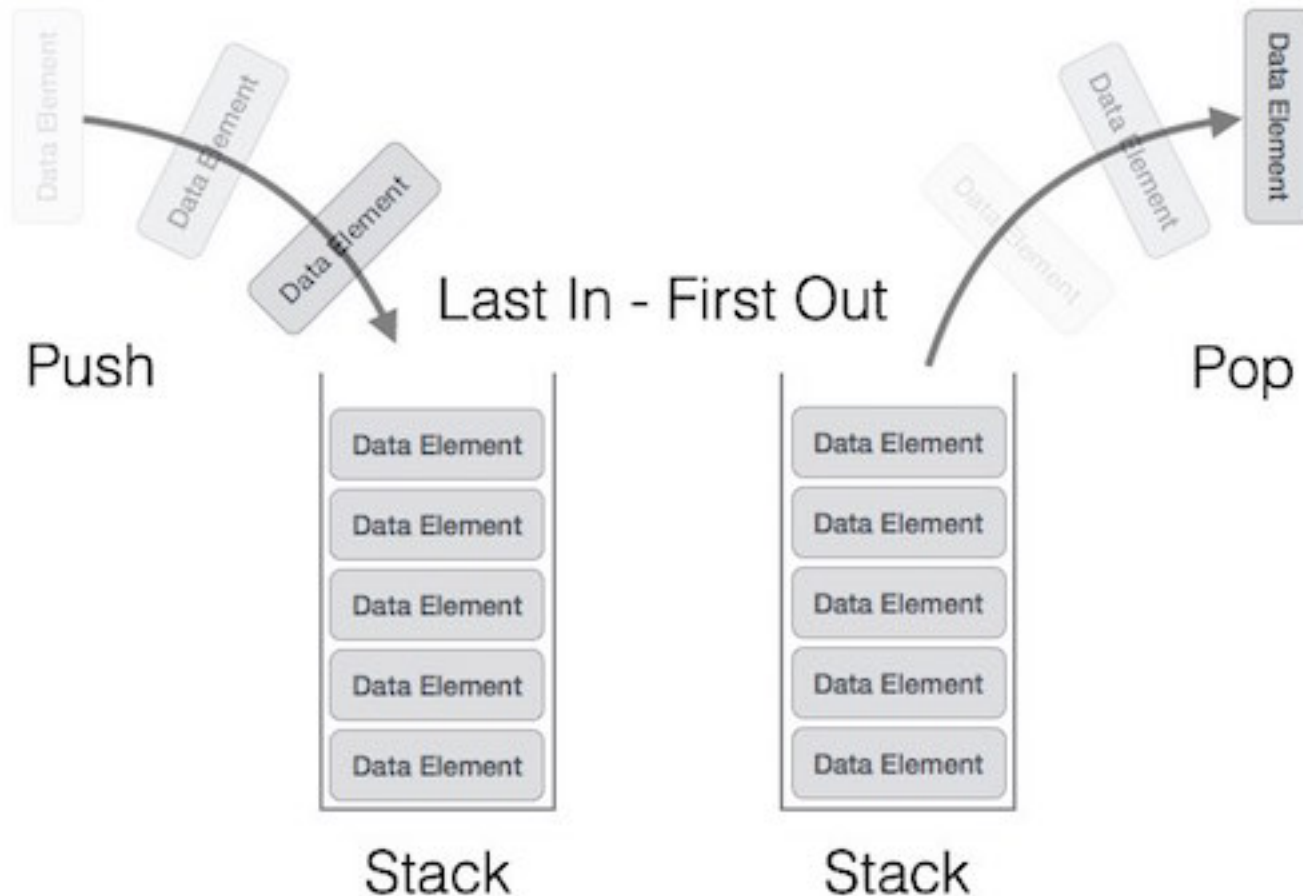
Stack

- A stack is an Abstract Data Type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example – a deck of cards or a pile of plates, etc.
- A real-world stack allows operations at one end only. For example, we can place or remove a card or plate from the top of the stack only. Likewise, Stack ADT allows all data operations at one end only. At any given time, we can only access the top element of a stack.
- This feature makes it LIFO data structure. LIFO stands for Last-in-first-out. Here, the element which is placed (inserted or added) last, is accessed first. In stack terminology, insertion operation is called **PUSH** operation and removal operation is called **POP** operation.

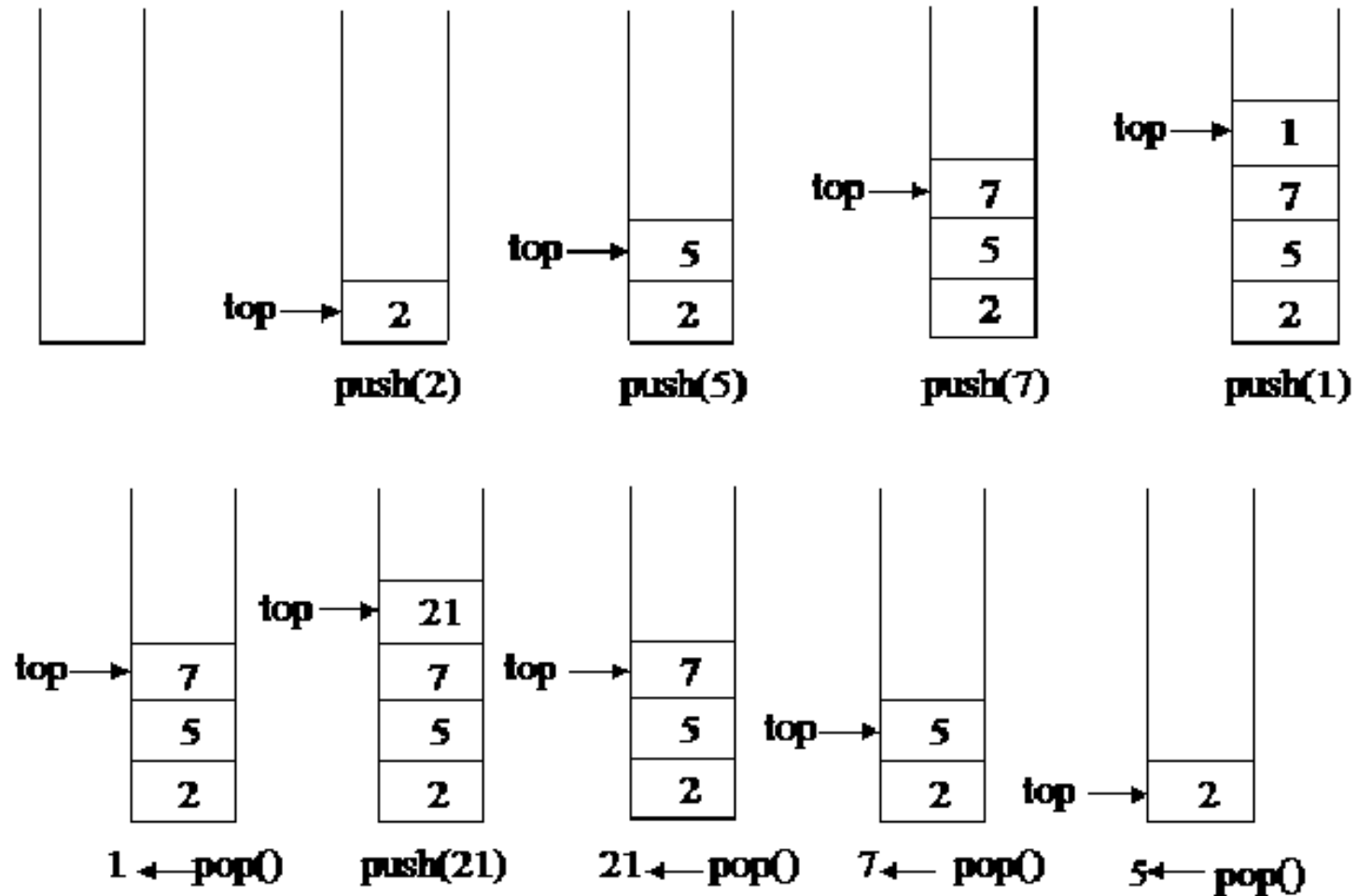


Stack Representation

➤ The following diagram depicts a stack and its operations –



Stack Representation



Basic Operations on Stack

- Stack operations may involve initializing the stack, using it and then de-initializing it. Apart from these basic stuffs, a stack is used for the following two primary operations –
 - **push()** – Pushing (storing) an element on the stack.
 - **pop()** – Removing (accessing) an element from the stack.
- To use a stack efficiently, we need to check the status of stack as well. For the same purpose, the following functionality is added to stacks –
 - **peek()** – get the top data element of the stack, without removing it.
 - **isFull()** – check if stack is full.
 - **isEmpty()** – check if stack is empty.
- At all times, we maintain a pointer to the last PUSHed data on the stack. As this pointer always represents the top of the stack, hence named **top**. The **top** pointer provides top value of the stack without actually removing it.

Stack Implementation

- Stack data structure can be implement in two ways. They are as follows...
 - **Using Array**
 - **Using Linked List**
- When stack is implemented using array, that stack can organize only limited number of elements. When stack is implemented using linked list, that stack can organize unlimited number of elements.



Stack Implementation using Array

Stack Operations using Array

- A stack can be implemented using array as follows...

Before implementing actual operations, first follow the below steps to create an empty stack.

- **Step 1:** Include all the **header files** which are used in the program and define a constant '**SIZE**' with specific value.
- **Step 2:** Declare all the **functions** used in stack implementation.
- **Step 3:** Create a one dimensional array with fixed size (**int stack[SIZE]**)
- **Step 4:** Define a integer variable '**top**' and initialize with '**-1**'. (**int top = -1**)
- **Step 5:** In main method display menu with list of operations and make suitable function calls to perform operation selected by the user on the stack.

peek() operation

➤ Algorithm of peek() function –

begin procedure peek

return stack[top]

end procedure

➤ Implementation of peek() function in C/C++ programming language

```
int peek() {  
    return stack[top];  
}
```

isfull() operation

➤ Algorithm of isfull() function –

begin procedure isfull

if top equals to MAXSIZE

return true

else

return false

endif

end procedure

isfull() operation

- Implementation of isfull() function in C/C++ programming language

```
bool isfull() {  
    if(top == MAXSIZE)  
        return true;  
    else  
        return false;  
}
```

isempty() operation

➤ Algorithm of isempty() function –

```
begin procedure isempty()
```

```
    if top less than 1
```

```
        return true
```

```
    else
```

```
        return false
```

```
    endif
```

```
end procedure
```

isempty() operation

- Implementation of isempty() function in C programming language is slightly different. We initialize top at -1, as the index in array starts from 0. So we check if the top is below zero or -1 to determine if the stack is empty.

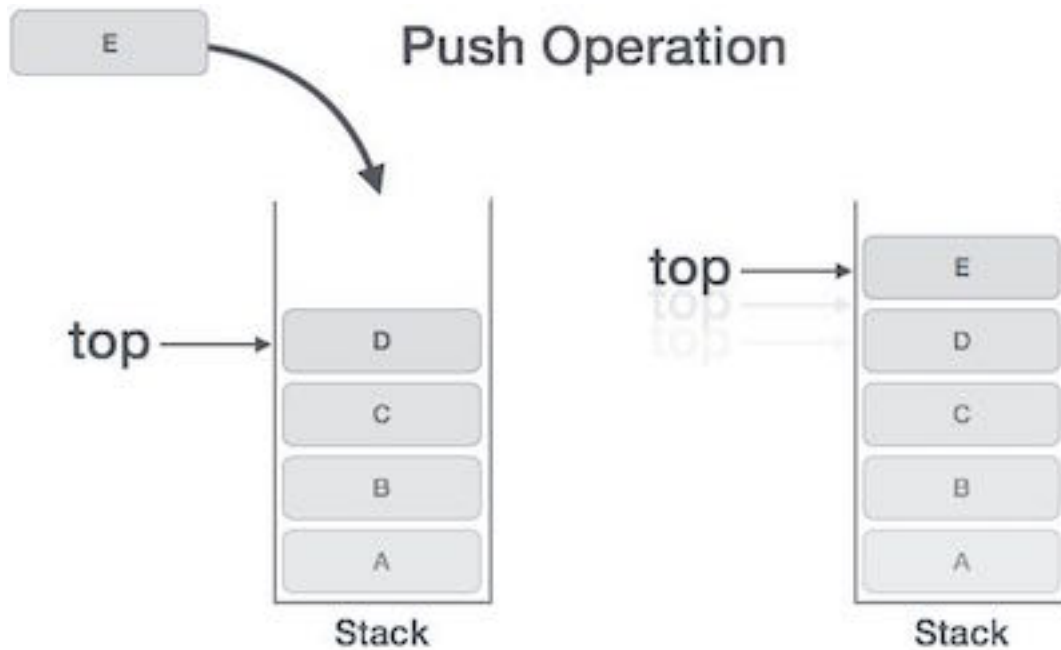
```
bool isempty() {  
    if(top == -1)  
        return true;  
    else  
        return false;  
}
```

Push Operation

➤ In a stack, `push()` is a function used to insert an element into the stack. In a stack, the new element is always inserted at **top** position. Push function takes one integer value as parameter and inserts that value into the stack. We can use the following steps to push an element on to the stack...

- **Step 1:** Check whether **stack** is **FULL**. (**top** == **SIZE-1**)
- **Step 2:** If it is **FULL**, then display "**Stack is FULL!!! Insertion is not possible!!!**" and terminate the function.
- **Step 3:** If it is **NOT FULL**, then increment **top** value by one (**top++**) and set **stack[top]** to value (**stack[top] = value**).

Push Operation





➤ Algorithm for Push() operation

begin procedure push: stack, data

 if stack is full

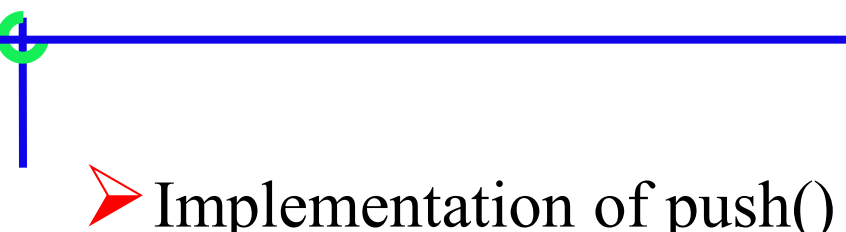
 return null

 endif

$\text{top} \leftarrow \text{top} + 1$

$\text{stack}[\text{top}] \leftarrow \text{data}$

end procedure

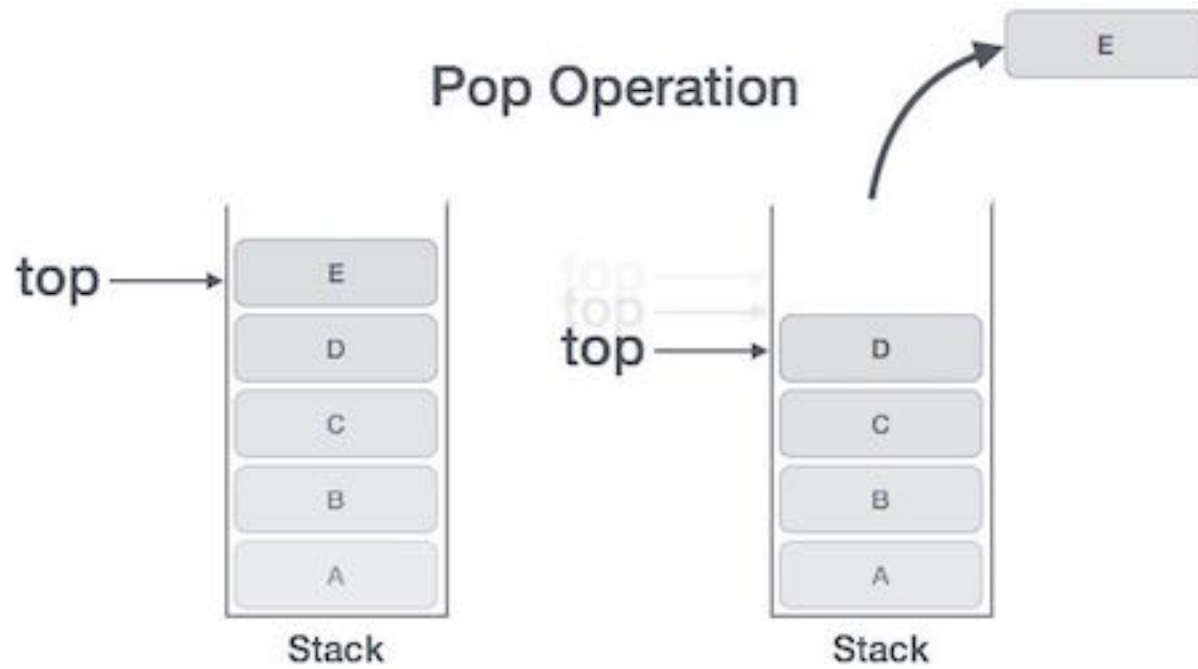


➤ Implementation of push() in C/C++

```
void push(int data) {  
    if(!isFull()) {  
        top = top + 1;  
        stack[top] = data;  
    } else {  
        printf("Could not insert data, Stack is full.\n");  
    }  
}
```

Pop Operation

- In a stack, `pop()` is a function used to delete an element from the stack. In a stack, the element is always deleted from **top** position. Pop function does not take any value as parameter. We can use the following steps to pop an element from the stack...
- **Step 1:** Check whether **stack** is **EMPTY**. (**top == -1**)
- **Step 2:** If it is **EMPTY**, then display "**Stack is EMPTY!!! Deletion is not possible!!!**" and terminate the function.
- **Step 3:** If it is **NOT EMPTY**, then delete **stack[top]** and decrement **top** value by one (**top--**).





➤ Algorithm for Pop() operation

```
begin procedure pop: stack
```

```
  if stack is empty
```

```
    return null
```

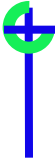
```
  endif
```

```
  data  $\leftarrow$  stack[top]
```

```
  top  $\leftarrow$  top - 1
```

```
  return data
```

```
end procedure
```



➤ Implementation of pop() in C/C++

```
int pop(int data) {  
  
    if(!isempty()) {  
        data = stack[top];  
        top = top - 1;  
        return data;  
    } else {  
        printf("Could not retrieve data, Stack is empty.\n");  
    }  
}
```

Display operation

- We can use the following steps to display the elements of a stack...
- **Step 1:** Check whether **stack** is **EMPTY**. (**top == -1**)
- **Step 2:** If it is **EMPTY**, then display "**Stack is EMPTY!!!**" and terminate the function.
- **Step 3:** If it is **NOT EMPTY**, then define a variable '**i**' and initialize with **top**. Display **stack[i]** value and decrement **i** value by one (**i--**).
- **Step 3:** Repeat above step until **i** value becomes '0'.



➤ Implementation of display() in C/C++

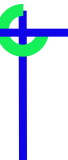
```
void display(){
    if(top == -1)
        printf("\nStack is Empty!!!");
    else{
        int i;
        printf("\nStack elements are:\n");
        for(i=top; i>=0; i--)
            printf("%d\n",stack[i]);
    }
}
```



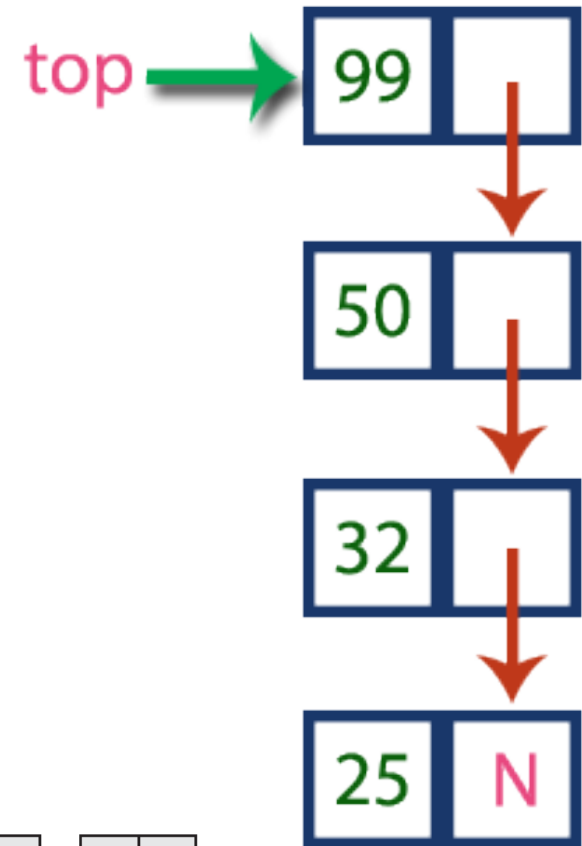
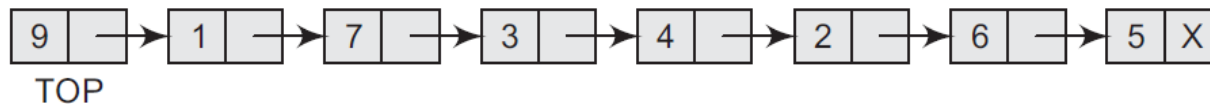

➤ Program on Stack using array



Stack Implementation using Linked List

- 
- The major problem with the stack implemented using array is, it works only for fixed number of data values. That means the amount of data must be specified at the beginning of the implementation itself. Stack implemented using array is not suitable, when we don't know the size of data which we are going to use.
 - A stack data structure can be implemented by using linked list data structure. The stack implemented using linked list can work for unlimited number of values. That means, stack implemented using linked list works for variable size of data. So, there is no need to fix the size at the beginning of the implementation. The Stack implemented using linked list can organize as many data values as we want.
 - In linked list implementation of a stack, every new element is inserted as '**top**' element. That means every newly inserted element is pointed by '**top**'. Whenever we want to remove an element from the stack, simply remove the node which is pointed by '**top**' by moving '**top**' to its next node in the list. The **next** field of the first element must be always **NULL**.

- In above example, the last inserted node is 99 and the first inserted node is 25. The order of elements inserted is 25, 32, 50 and 99.



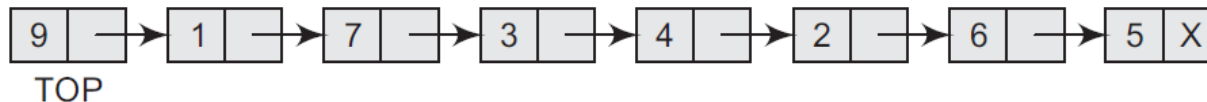
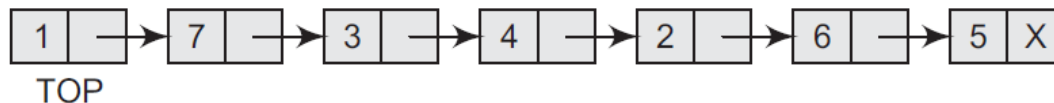
Stack Operations using Linked List

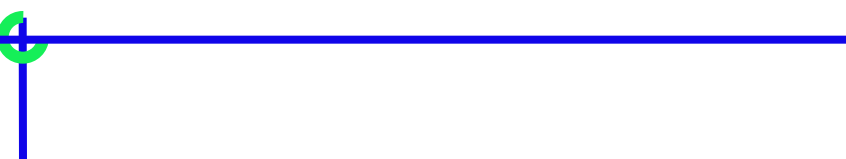
- To implement stack using linked list, we need to set the following things before implementing actual operations.
- **Step 1:** Include all the **header files** which are used in the program. And declare all the **user defined functions**.
- **Step 2:** Define a '**Node**' structure with two members **data** and **next**.
- **Step 3:** Define a **Node** pointer '**top**' and set it to **NULL**.
- **Step 4:** Implement the **main** method by displaying Menu with list of operations and make suitable function calls in the **main** method.

Push Operation

➤ We can use the following steps to insert a new node into the stack...

- **Step 1:** Create a **newNode** with given value.
- **Step 2:** Check whether stack is **Empty** (**top == NULL**)
- **Step 3:** If it is **Empty**, then set **newNode** $\rightarrow$ **next = NULL**.
- **Step 4:** If it is **Not Empty**, then set **newNode** $\rightarrow$ **next = top**.
- **Step 5:** Finally, set **top = newNode**.



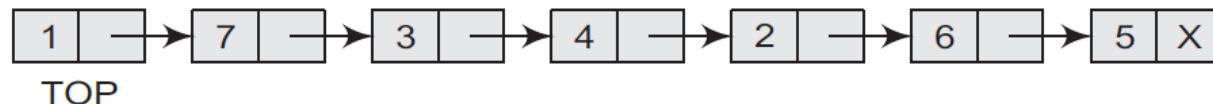
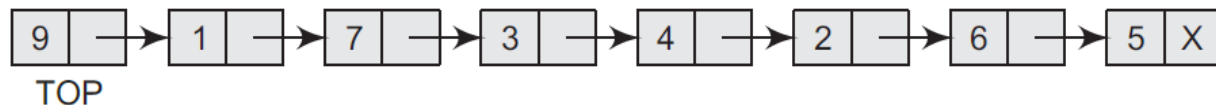


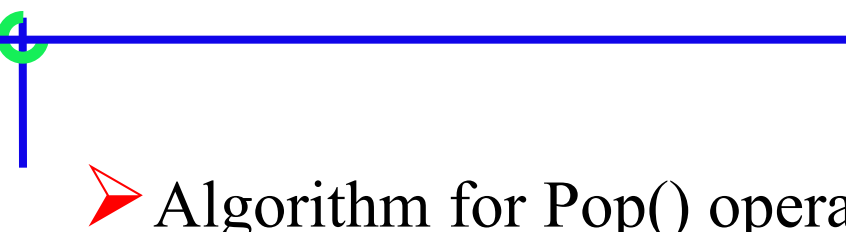
➤ Algorithm for Push() operation

```
Step 1: Allocate memory for the new
        node and name it as NEW_NODE
Step 2: SET NEW_NODE -> DATA = VAL
Step 3: IF TOP = NULL
        SET NEW_NODE -> NEXT = NULL
        SET TOP = NEW_NODE
    ELSE
        SET NEW_NODE -> NEXT = TOP
        SET TOP = NEW_NODE
    [END OF IF]
Step 4: END
```

Pop Operation

- We can use the following steps to delete a node from the stack...
- **Step 1:** Check whether **stack** is **Empty** (**top == NULL**).
 - **Step 2:** If it is **Empty**, then display "**Stack is Empty!!!**
Deletion is not possible!!!" and terminate the function
 - **Step 3:** If it is **Not Empty**, then define a **Node** pointer '**temp**' and set it to '**top**'.
 - **Step 4:** Then set '**top = top → next**'.
 - **Step 7:** Finally, delete '**temp**' (**free(temp)**).





➤ Algorithm for Pop() operation

```
Step 1: IF TOP = NULL
        PRINT "UNDERFLOW"
        Goto Step 5
    [END OF IF]
Step 2: SET PTR = TOP
Step 3: SET TOP = TOP -> NEXT
Step 4: FREE PTR
Step 5: END
```

Display operation

➤ We can use the following steps to display the elements (nodes) of a stack...

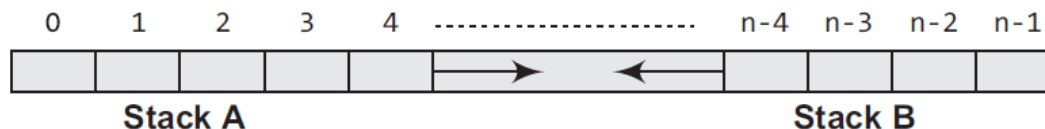
- **Step 1:** Check whether stack is **Empty** (**top == NULL**).
- **Step 2:** If it is **Empty**, then display '**Stack is Empty!!!**' and terminate the function.
- **Step 3:** If it is **Not Empty**, then define a Node pointer '**temp**' and initialize with **top**.
- **Step 4:** Display '**temp → data --->**' and move it to the next node. Repeat the same until **temp** reaches to the first node in the stack (**temp → next != NULL**).
- **Step 4:** Finally! Display '**temp → data ---> NULL**'.

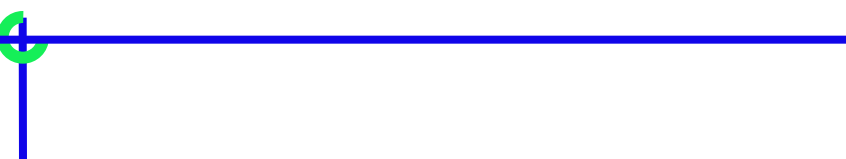


➤ Program on Stack using linked list

Multiple Stack

- While implementing a stack using an array, we had seen that the size of the array must be known in advance. If the stack is allocated less space, then frequent OVERFLOW conditions will be encountered.
- To deal with this problem, the code will have to be modified to reallocate more space for the array. In case we allocate a large amount of space for the stack, it may result in sheer wastage of memory.
- Thus, there lies a trade-off between the frequency of overflows and the space allocated.
- So, a better solution to deal with this problem is to have multiple stacks or to have more than one stack in the same array of sufficient size.



- 
- An array `STACK[n]` is used to represent two stacks, Stack A and Stack B. The value of `n` is such that the combined size of both the stacks will never exceed `n`.
 - While operating on these stacks, it is important to note one thing—Stack A will grow from left to right, whereas Stack B will grow from right to left at the same time.
 - Program on multiple Stack.



Applications of Stacks

Applications of Stacks

- 
- Stacks can be used on many applications.
 - Conversion of an infix expression into a postfix expression
 - Evaluation of a postfix expression
 - Conversion of an infix expression into a prefix expression
 - Evaluation of a prefix expression
 - Reversing a list
 - Parentheses checker
 - Recursion
 - Tower of Hanoi



Expressions

Expressions

- In any programming language, if we want to perform any calculation or to frame a condition etc., we use a set of symbols to perform the task. These set of symbols makes an expression.
- An expression can be defined as follows...
 - **An expression is a collection of operators and operands that represents a specific value.**

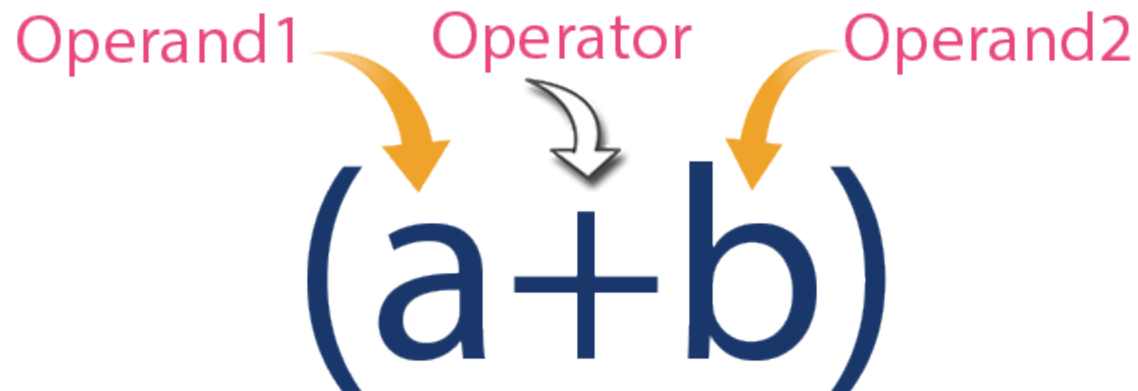
Expression Types

➤ Based on the operator position, expressions are divided into THREE types. They are as follows...

- **Infix Expression**
- **Postfix Expression**
- **Prefix Expression**

Infix Expression

- In infix expression, operator is used in between operands.
- The general structure of an Infix expression is as follows...



Postfix Expression

- In postfix expression, operator is used after operands. We can say that "**Operator follows the Operands**".
- The general structure of Postfix expression is as follows...

Operand1 Operand2 Operator

 ↘ ↘ ↙

 a b +

The diagram illustrates the structure of a postfix expression. It shows the expression 'ab+' in large blue font. Above the 'a' is the label 'Operand1' in pink, with a yellow curved arrow pointing down to 'a'. Above the 'b' is the label 'Operand2' in pink, with a yellow curved arrow pointing down to 'b'. Above the '+' is the label 'Operator' in pink, with a white curved arrow pointing down to '+'. The labels are positioned above their respective parts of the expression, and the arrows indicate the flow from the labels to the expression components.

Prefix Expression

- In prefix expression, operator is used before operands. We can say that "**Operands follows the Operator**".
- The general structure of Prefix expression is as follows...

Operator Operand1 Operand2

+ab

The diagram illustrates the structure of a prefix expression. It shows the expression '+ab' in large blue font. Above the expression, three labels are positioned: 'Operator' above the '+', 'Operand1' above the 'a', and 'Operand2' above the 'b'. Three curved arrows point from these labels to their respective parts in the expression: a white arrow from 'Operator' to '+', and two yellow arrows from 'Operand1' and 'Operand2' to 'a' and 'b' respectively.

Precedence

➤ When an operand is in between two different operators, which operator will take the operand first, is decided by the precedence of an operator over others.

➤ For example –

$a + b * c \rightarrow a + (b * c)$

➤ As multiplication operation has precedence over addition, $b * c$ will be evaluated first. A table of operator precedence is provided later.

Precedence



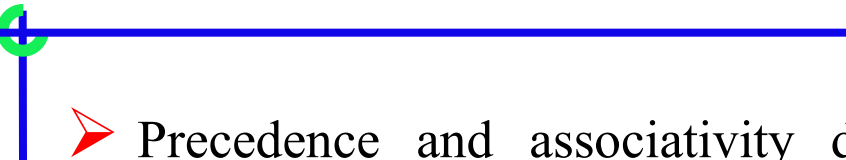
➤ For example,

$$x = 7 + 3 * 2;$$

➤ Here, x is assigned 13, not 20 because operator * has a higher precedence than +, so it first gets multiplied with 3*2 and then adds into 7.

Associativity

- Associativity describes the rule where operators with the same precedence appear in an expression.
- For example, in expression $a + b - c$, both $+$ and $-$ have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators.
- Here, both $+$ and $-$ are left associative, so the expression will be evaluated as $(a + b) - c$.



➤ Precedence and associativity determines the order of evaluation of an expression.

➤ Following is an operator precedence and associativity table (highest to lowest)

Sr. No.	Operator	Precedence	Associativity
1	Exponentiation $^$	Highest	Right Associative
2	Multiplication ($*$) & Division ($/$)	Second Highest	Left Associative
3	Addition ($+$) & Subtraction ($-$)	Lowest	Left Associative

➤ Complete Precedence list

Level	Operators	Description	Associativity
15	() [] . new	Function Call Array Subscript Object Property Access Memory Allocation	Left to Right
14	++ -- + - ! ~ delete typeof void	Increment / Decrement Unary plus / minus Logical negation / bitwise complement Deallocation Find type of variable	Right to Left
13	* / %	Multiplication Division Modulo	Left to Right
12	+ -	Addition / Subtraction	Left to Right
11	>> <<	Bitwise Right Shift Bitwise Left Shift	Left to Right
10	< <= > >=	Relational Less Than / Less than Equal To Relational Greater / Greater than Equal To	Left to Right
9	== != === !==	Equality Inequality Identity Operator Non Identity Operator	Left to Right
8	&	Bitwise AND	Left to Right
7	^	Bitwise XOR	Left to Right
6		Bitwise OR	Left to Right
5	&&	Logical AND	Left to Right
4		Logical OR	Left to Right
3	?:	Conditional Operator	Right to Left
2	= += -= *= /= %= &= ^= = <<= >>=	Assignment Operators	Right to Left
1	,	Comma Operator	Left to Right

- 
- The above table shows the default behavior of operators. At any point of time in expression evaluation, the order can be altered by using parenthesis. For example –
 - In $a + b * c$, the expression part $b * c$ will be evaluated first, with multiplication as precedence over addition.
 - But if we use parenthesis for $a + b$. Then expression will be evaluated first, like $(a + b) * c$.

Expression Conversion

➤ To convert any Infix expression into Postfix or Prefix expression we can use the following procedure...

- Find all the operators in the given Infix Expression.
- Find the order of operators evaluated according to their Operator precedence.
- Convert each operator into required type of expression (Postfix or Prefix) in the same order.

- 
- Any expression can be represented using these three different types of expressions.
 - And we can convert an expression from one form to another form like
 - **Infix to Postfix,**
 - **Infix to Prefix,**
 - **Prefix to Postfix**
 - **Prefix to infix**
 - **Postfix to prefix**
 - **Postfix to infix**

➤ Example

Consider the following Infix Expression to be converted into Postfix Expression...

$$D = A + B * C$$

- **Step 1:** The Operators in the given Infix Expression : = , + , *
- **Step 2:** The Order of Operators according to their preference : * , + , =
- **Step 3:** Now, convert the first operator * ----- $D = A + B C *$
- **Step 4:** Convert the next operator + ----- $D = A B C * +$
- **Step 5:** Convert the next operator = ----- $D A B C * + =$

Finally, given Infix Expression is converted into Postfix Expression as follows...

$$D A B C * + =$$

Exercise

➤ Convert the following infix expressions into postfix and prefix expressions

■ $(A-B)*(C+D)$

$$[AB-] * [CD+]$$

$$AB-CD+*$$

$$[-AB] * [+CD]$$

$$*-AB+CD$$

Exercise

➤ Convert the following infix expressions into postfix and prefix expressions

■ $(A+B)/(C+D)-(D * E)$

$$[AB+] / [CD+] - [DE*]$$

$$[AB+CD+/-] - [DE*]$$

$$AB+CD+ / DE* -$$

$$[+AB] / [+CD] - [*DE]$$

$$[/+AB+CD] - [*DE]$$

$$- / + AB + CD * DE$$

➤ Convert the following infix expressions into prefix expressions.

■ $(A + B) * C$

$$(+AB)*C$$

$$*+ABC$$

■ $(A-B) * (C+D)$

$$[-AB] * [+CD]$$

$$*-AB+CD$$

■ $(A + B) / (C + D) - (D * E)$

$$[+AB] / [+CD] - [*DE]$$

$$[/+AB+CD] - [*DE]$$

$$-/+AB+CD*DE$$

Infix to Postfix Conversion

➤ Procedure for Postfix Conversion

- Scan the Infix string from left to right.
- Initialize an empty stack.
- If the scanned character is an operand, add it to the Postfix string.
- If the scanned character is an operator and if the stack is empty push the character to stack.
- If the scanned character is an Operator and the stack is not empty, compare the precedence of the character with the element on top of the stack.
- If top Stack has higher precedence over the scanned character pop the stack else push the scanned character to stack. Repeat this step until the stack is not empty and top Stack has precedence over the character.
- Repeat 4 and 5 steps till all the characters are scanned.
- After all characters are scanned, we have to add any character that the stack may have to the Postfix string.
- If stack is not empty add top Stack to Postfix string and Pop the stack.
- Repeat this step as long as stack is not empty.



➤ Algorithm for Postfix Conversion

S:stack

while(more tokens)

$x \leq$ next token

 if($x ==$ operand)

 print x

 else

 while($\text{precedence}(x) \leq \text{precedence}(\text{top}(s))$)

 print(pop(s))

 push(s,x)

 while(! empty (s))

 print(pop(s))

Conversion To Postfix

$$A+(B*C-(D/E-F)*G)*H$$

Input	Stack	Output
A+(B*C-(D/E-F)*G)*H	Empty	-
+(B*C-(D/E-F)*G)*H	Empty	A
(B*C-(D/E-F)*G)*H	+	A
B*C-(D/E-F)*G)*H	+(	A
*C-(D/E-F)*G)*H	+(	AB
C-(D/E-F)*G)*H	+(*	AB
-(D/E-F)*G)*H	+(*	ABC
(D/E-F)*G)*H	+(-	ABC*
D/E-F)*G)*H	+(-(	ABC*

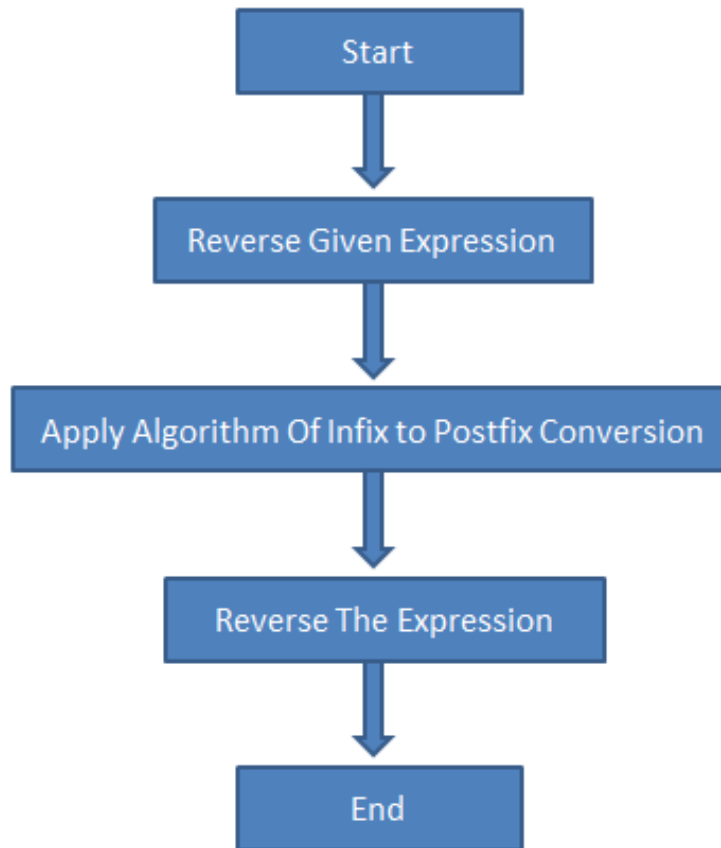
/E-F)*G)*H	+(-(	ABC*D
E-F)*G)*H	+(-(/	ABC*D
-F)*G)*H	+(-(/	ABC*DE
F)*G)*H	+(-(-	ABC*DE/
F)*G)*H	+(-(-	ABC*DE/
)*G)*H	+(-(-	ABC*DE/F
*G)*H	+(-	ABC*DE/F-
G)*H	+(-*	ABC*DE/F-
)*H	+(-*	ABC*DE/F-G
*H	+	ABC*DE/F-G*-
H	+	ABC*DE/F-G*-
End	+	ABC*DE/F-G*-H
End	Empty	ABC*DE/F-G*-H*+

$$(A - (B / C + (D \% E * F) / G) * H)$$

Infix Character Scanned	Stack	Postfix Expression
	(	
A	(	A
-	(-	A
(	(- (	A
B	(- (	A B
/	(- (/	A B
C	(- (/	A B C
+	(- (+	A B C /
(	(- (+ (	A B C /
D	(- (+ (	A B C / D
%	(- (+ (%	A B C / D
E	(- (+ (%	A B C / D E
*	(- (+ (% *	A B C / D E
F	(- (+ (% *	A B C / D E F
)	(- (+	A B C / D E F * %
/	(- (+ /	A B C / D E F * %
G	(- (+ /	A B C / D E F * % G
)	(-	A B C / D E F * % G / +
*	(- *	A B C / D E F * % G / +
H	(- *	A B C / D E F * % G / + H
)		A B C / D E F * % G / + H * -

Infix to Prefix Conversion

➤ Algorithm of Infix to Prefix



- 
- Step 1: Reverse the infix string. Note that while reversing the string you must interchange left and right parentheses.
 - Step 2: Obtain the postfix expression of the infix expression obtained in Step 1.
 - Step 3: Reverse the postfix expression to get the prefix expression



➤ Infix to prefix conversion

➤ Expression = $(A+B^C)*D+E^5$



Step 1. Reverse the infix expression.

$5^E+D^*)C^B+A($

Step 2. Make Every '(' as ')' and every ')' as '('

$5^E+D^*(C^B+A)$

Step 3. Convert expression to postfix form.



Expression	Stack	Output
$5^E + D * (C^B + A)$	Empty	-
$^E + D * (C^B + A)$	Empty	5
$E + D * (C^B + A)$	^	5
$+ D * (C^B + A)$	^	5E
$D * (C^B + A)$	+	5E^
$*(C^B + A)$	+	5E^D
$(C^B + A)$	+*	5E^D
$C^B + A)$	+*(	5E^D
$^B + A)$	+*(	5E^DC



B+A)	+*(^	5E^DC
+A)	+*(^	5E^DCB
A)	+*(+	5E^DCB^
)	+*(+	5E^DCB^A
End	+	5E^DCB^A+
End	Empty	5E^DCB^A+*+

Step 4. Reverse the expression.

+*+A^BCD^E5

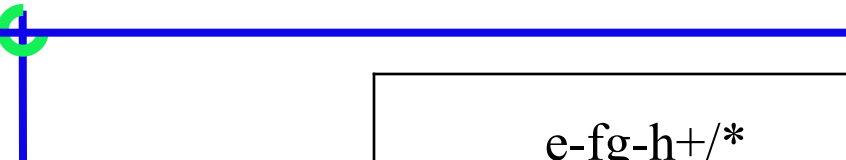
Postfix to Infix Conversion

➤ Algorithm of Postfix to Infix

- 1. While there are input symbol left
- 2. Read the next symbol from input.
- 3. If the symbol is an operand
 - Push it onto the stack.
- 4. Otherwise, the symbol is an operator.
- 5. If there are fewer than 2 values on the stack
 - Show Error /* input not sufficient values in the expression */
- 6. Else
 - Pop the top 2 values from the stack.
 - Put the operator, with the values as arguments and form a string.
 - Encapsulate the resulted string with parenthesis.
 - Push the resulted string back to stack.
- 7. If there is only one value in the stack
 - That value in the stack is the desired infix string.
- 8. If there are more values in the stack
 - Show Error /* The user input has too many values */

abc-+de-fg-h+/*

Expression	Stack
abc-+de-fg-h+/*	NuLL
bc-+de-fg-h+/*	"a"
c-+de-fg-h+/*	"b" "a"
-+de-fg-h+/*	"c" "b" "a"
+de-fg-h+/*	"b - c" "a"
de-fg-h+/*	"a+b-c"



e-fg-h+/*	"d" "a+b-c"
-fg-h+/*	"e" "d" "a+b-c"
fg-h+/*	"d - e" "a+b-c"
g-h+/*	"f" "d - e" "a+b-c"
-h+/*	"g" "f" "d - e" "a+b-c"
h+/*	"f-g" "d - e" "a+b-c"



$+/*$	$"h"$ $"f-g"$ $"d - e"$ $"a+b-c"$
$/*$	$"f-g+h"$ $"d - e"$ $"a+b-c"$
$*$	$"(d-e)/(f-g-h)"$ $"a+b-c"$
Null	$"(a+b-c)*(d-e)/(f-g+h)"$

Prefix to Infix Conversion

➤ Algorithm of Prefix to Infix

- This algorithm is a non-tail recursive method.
- 1. The reversed input string is completely pushed into a stack.
- `prefixToInfix(stack)`
- 2. IF stack is not empty
 - a. Temp -->pop the stack
 - b. IF temp is a operator
 - Write a opening parenthesis to output
 - `prefixToInfix(stack)`
 - Write temp to output
 - `prefixToInfix(stack)`
 - Write a closing parenthesis to output
 - c. ELSE IF temp is a space -->`prefixToInfix(stack)`
 - d. ELSE
 - Write temp to output
 - IF `stack.top` NOT EQUAL to space -->`prefixToInfix(stack)`

*+a-bc/-de+-fgh

Expression	Stack
*+a-bc/-de+-fgh	NuLL
*+a-bc/-de+-fg	"h"
*+a-bc/-de+-f	"g" "h"
*+a-bc/-de+-	"f" "g" "h"
*+a-bc/-de+	"f - g" "h"
*+a-bc/-de	"f-g+h"



$*+a-bc/-d$	"e" "f-g+h"
$*+a-bc/-$	"d" "e" "f-g+h"
$*+a-bc/$	"d - e" "f-g+h"
$*+a-bc$	"(d-e)/(f-g+h)"
$*+a-b$	"c" "(d-e)/(f-g+h)"



$*+a-$	$"b"$ $"c"$ $"(d-e)/(f-g+h)"$
$*+a$	$"b-c"$ $"(d-e)/(f-g+h)"$
$*+$	$"a"$ $"b-c"$ $"(d-e)/(f-g+h)"$
$*$	$"a+b-c"$ $"(d-e)/(f-g+h)"$
End	$"(a+b-c)*(d-e)/(f-g+h)"$

Postfix Expression Evaluation

- A postfix expression can be evaluated using the Stack data structure.
- To evaluate a postfix expression using Stack data structure we can use the following steps...
 - Read all the symbols one by one from left to right in the given Postfix Expression
 - If the reading symbol is operand, then push it on to the Stack.
 - If the reading symbol is operator (+ , - , * , / etc.), then perform TWO pop operations and store the two popped operands in two different variables (operand1 and operand2). Then perform reading symbol operation using operand1 and operand2 and push result back on to the Stack.
 - Finally! perform a pop operation and display the popped value as final result.

➤ Consider the infix expression given as $9 - ((3 * 4) + 8) / 4$.

➤ The infix expression $9 - ((3 * 4) + 8) / 4$ can be written as $9\ 3\ 4\ *\ 8\ +\ 4\ /\ -$ using postfix notation.

Character Scanned	Stack
9	9
3	9, 3
4	9, 3, 4
*	9, 12
8	9, 12, 8
+	9, 20
4	9, 20, 4
/	9, 5
-	4

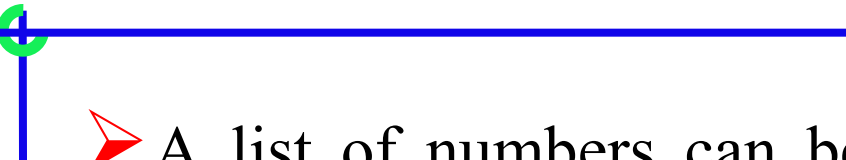
Prefix Expression Evaluation

- The algorithm for evaluating a prefix expression is as follows:
 - Accept a prefix string from the user
 - Repeat until all the characters in the prefix expression have been scanned
 - Scan the prefix expression from right, one character at a time.
 - If the scanned character is an operand, push it on the operand stack
 - If the scanned character is an operator, then
 - (i). Pop two values from the operand stack
 - (ii). Apply the operator on the popped operands
 - (iii). Push the result on the operand stack
 - End

➤ For example, consider the prefix expression $+ - 9 2 7 * 8 / 4 12$.

Character scanned	Operand stack
12	12
4	12, 4
/	3
8	3, 8
*	24
7	24, 7
2	24, 7, 2
-	24, 5
+	29

Reversing a List

- 
- A list of numbers can be reversed by reading each number from an array starting from the first index and pushing it on a stack. Once all the numbers have been read, the numbers can be popped one at a time and then stored in the array starting from the first index.

Implementing Parentheses Checker

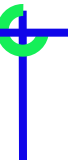
- Stacks can be used to check the validity of parentheses in any algebraic expression. For example, an algebraic expression is valid if for every open bracket there is a corresponding closing bracket.
- For example, the expression $(A+B\}$ is invalid but an expression $\{A + (B - C)\}$ is valid. Look at the program below which traverses an algebraic expression to check for its validity.

➤ **Algorithm:**

- 1) Declare a character stack S.
- 2) Now traverse the expression string exp.
 - a) If the current character is a starting bracket ('(' or '{' or '[') then push it to stack.
 - b) If the current character is a closing bracket (')' or '}' or ']') then pop from stack and if the popped character is the matching starting bracket then fine else parenthesis are not balanced.
- 3) After complete traversal, if there is some starting bracket left in stack then “not balanced”

Recursion

- A *recursive function* is defined as a function that calls itself to solve a smaller version of its task until a final call is made which does not require a call to itself.
- Since a recursive function repeatedly calls itself, it makes use of the system stack to temporarily store the return address and local variables of the calling function.
- Every recursive solution has two major cases. They are
 - **Base case**, in which the problem is simple enough to be solved directly without making any further calls to the same function.
 - **Recursive case**, in which first the problem at hand is divided into simpler sub-parts. Second the function calls itself but with sub-parts of the problem obtained in the first step. Third, the result is obtained by combining the solutions of simpler sub-parts.
- Therefore, recursion is defining large and complex problems in terms of smaller and more easily solvable problems. In recursive functions, a complex problem is defined in terms of simpler problems and the simplest problem is given explicitly.

- 
- To understand recursive functions, let us take an example of calculating factorial of a number.
 - To calculate $n!$, we multiply the number with factorial of the number that is 1 less than that number.
 - In other words, $n! = n \times (n-1)!$
 - Let us say we need to find the value of $5!$
 - $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
 - This can be written as $5! = 5 \times 4!$, where $4! = 4 \times 3!$
 - Therefore, $5! = 5 \times 4 \times 3!$
 - Similarly, we can also write $5! = 5 \times 4 \times 3 \times 2!$
 - Expanding further $5! = 5 \times 4 \times 3 \times 2 \times 1!$
 - We know, $1! = 1$

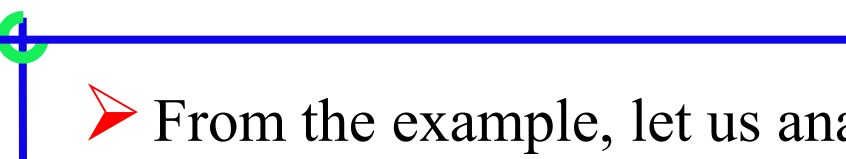
PROBLEM	SOLUTION
$5!$	$5 \times 4 \times 3 \times 2 \times 1!$
$= 5 \times 4!$	$= 5 \times 4 \times 3 \times 2 \times 1$
$= 5 \times 4 \times 3!$	$= 5 \times 4 \times 3 \times 2$
$= 5 \times 4 \times 3 \times 2!$	$= 5 \times 4 \times 6$
$= 5 \times 4 \times 3 \times 2 \times 1!$	$= 5 \times 24$
	$= 120$



➤ every recursive function must have a base case and a recursive case.

For the factorial function

- **Base case** is when $n = 1$, because if $n = 1$, the result will be 1 as $1! = 1$.
- **Recursive case** of the factorial function will call itself but with a smaller value of n , this case can be given as $\text{factorial}(n) = n \times \text{factorial}(n-1)$



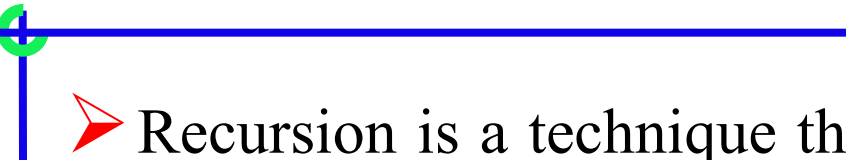
➤ From the example, let us analyze the steps of a recursive program.

- *Step 1:* Specify the base case which will stop the function from making a call to itself.
- *Step 2:* Check to see whether the current value being processed matches with the value of the base case. If yes, process and return the value.
- *Step 3:* Divide the problem into smaller or simpler sub-problems.
- *Step 4:* Call the function from each sub-problem.
- *Step 5:* Combine the results of the sub-problems.
- *Step 6:* Return the result of the entire problem.

Examples of Recursion

- 
- Greatest Common Divisor
 - Finding Exponents
 - The Fibonacci Series

Types of Recursion

- 
- Recursion is a technique that breaks a problem into one or more sub-problems that are similar to the original problem. Any recursive function can be characterized based on
 - whether the function calls itself directly or indirectly (*direct or indirect recursion*),



➤ *Direct Recursion*

- A function is said to be *directly* recursive if it explicitly calls itself. For example, consider the code shown in following figure. Here, the function Func() calls itself for all positive values of n, so it is said to be a directly recursive function.

```
int Func (int n)
{
    if (n == 0)
        return n;
    else
        return (Func (n-1));
}
```



➤ *Indirect Recursion*

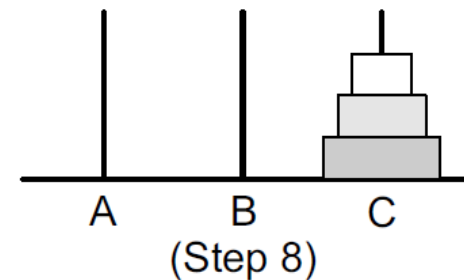
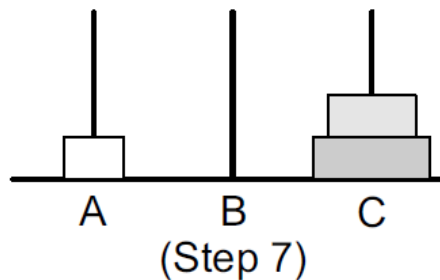
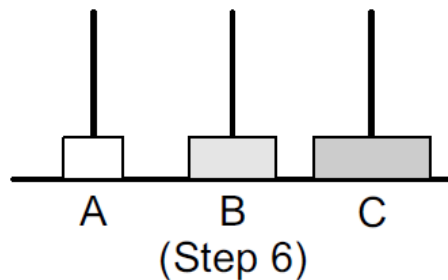
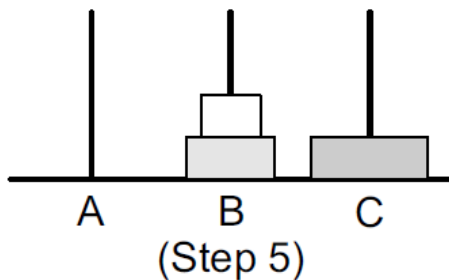
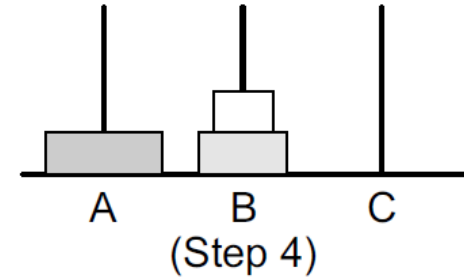
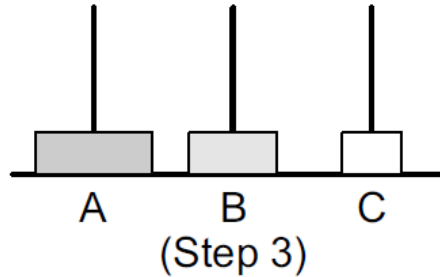
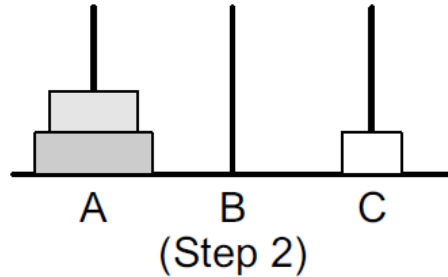
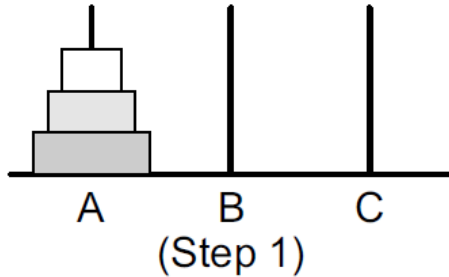
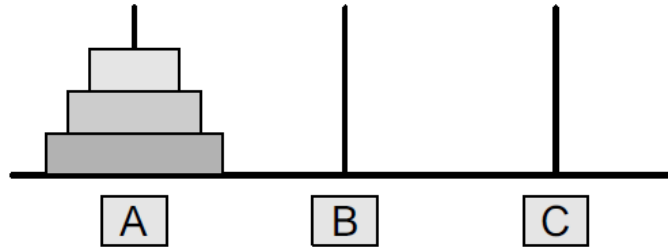
- A function is said to be *indirectly* recursive if it contains a call to another function which ultimately calls it. Look at the functions given below. These two functions are indirectly recursive as they both call each other.

```
int Func1 (int n)
{
    if (n == 0)
        return n;
    else
        return Func2(n);
}
int Func2(int x)
{
    return Func1(x-1);
}
```

Tower of Hanoi

- 
- The tower of Hanoi is one of the main applications of recursion. It says, ‘if you can solve $n-1$ cases, then you can easily solve the n th case’.

Tower of Hanoi



Advantage of Recursion

- Recursive solutions often tend to be shorter and simpler than non-recursive ones.
- Code is clearer and easier to use.
- Recursion works similar to the original formula to solve a problem.
- Recursion follows a divide and conquer technique to solve problems.
- In some (limited) instances, recursion may be more efficient.

Advantages/Drawback of recursion

- For some programmers and readers, recursion is a difficult concept.
- Recursion is implemented using system stack. If the stack space on the system is limited, recursion to a deeper level will be difficult to implement.
- Aborting a recursive program in midstream can be a very slow process.
- Using a recursive function takes more memory and time to execute as compared to its non-recursive counterpart.
- It is difficult to find bugs, particularly while using global variables.